# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

| | | | |
|---|---|---|---|
| **Course Code:** | CSA06 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO5– Naturalization (BL5,BL6) | **Assessment:** | Assessment Tool 1 – Constraint Based Problem Solving |
| **Weightage:** | 30% of CIA | **Total Marks:** | 100 |
| **CO5 Statement:** | *Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.* | | |
| **Student Name:** | CH.POORNA TEJA | **Reg. No.:** | 192472149 |
| **Section / Batch:** | | **Date:** | |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

| Question Type | Focus Area | Questions |
|---|---|---|
| Constraint Based Problem Solving | Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete. | Q1 |

## SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

**Knight's Tour Problem (Backtracking / Constraint Satisfaction Problem):**

**A knight must visit every square of a chessboard exactly once. Clearly define constraints related to valid knight moves and non-repetition. Analyze how these constraints reduce possible moves at each step. Develop a backtracking-based solution to explore paths. Apply pruning to eliminate dead-end paths and justify why the solution achieves completeness and discuss complexity.**

**Code of Conduct**

*I P . Sai Charan Tej(192465048)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understandi ng; misses key constraints | Fails to identify constraints correctly |
| Constraint Analysis | 25% | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritizatio n | Limited analysis; weak prioritization | No meaningful analysis or prioritization |
| Solution Development | 25% | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts |
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided |

## Marks Evaluation:

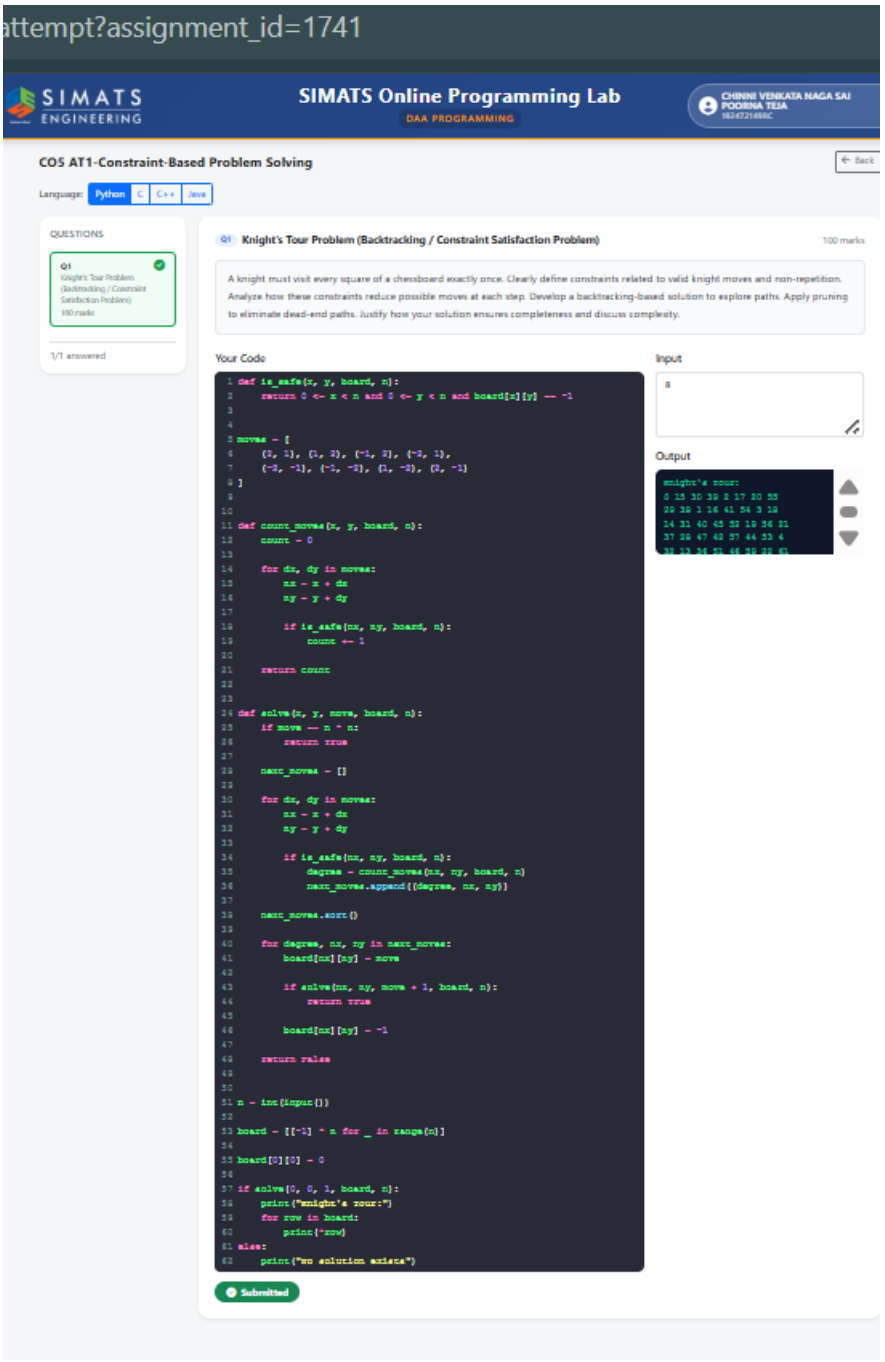
| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding | 25% | | | | |
| Constraint Analysis | 25% | | | | |
| Solution Development | 25% | | | | |
| Application of Concepts | 15% | | | | |
| Justification | 10% | | | | |
| **Total Marks (100):** | | | | | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| Name: | Name: | Name: |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

**Algorithm – Knight's Tour Using Backtracking**

1. **Start**
2. Read the size n of the chessboard.
3. Create an n × n board and initialize all cells to -1 to indicate that they are unvisited.
4. Place the knight at the starting position (0, 0) and mark it as move 0.
5. Define the **8 possible knight moves**:
    - (+2, +1)
    - (+2, -1)
    - (-2, +1)
    - (-2, -1)
    - (+1, +2)
    - (+1, -2)
    - (-1, +2)
    - (-1, -2)
6. For the current position, generate all possible next positions using these eight moves.
7. **Check constraints** for each candidate position:
    - The position must be inside the chessboard.
    - The position must not have been visited before.
8. If a valid position is found:
    - Mark it with the current move number.
    - Recursively continue the search from that position.
9. If all $n^2$ squares have been visited, a complete Knight's Tour is found.
10. If no valid move is possible:
    - **Backtrack** by resetting the current square to -1.
    - Return to the previous position and try another possible move.
11. Continue until a complete tour is found or all possibilities are exhausted.
12. Display the chessboard containing the move numbers.
13. **Stop.**

OUTPUT:


attempt?assignment_id=1741
SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
CHINNI VENKATA NAGA SAI POORNA TEJA
COS AT1-Constraint-Based Problem Solving
Back
Language: Python C C++ Java
QUESTIONS
Q1
Knight's Tour Problem (Backtracking / Constraint Satisfaction Problem)
100 marks
1/1 answered
Q1 Knight's Tour Problem (Backtracking / Constraint Satisfaction Problem)
100 marks
A knight must visit every square of a chessboard exactly once. Clearly define constraints related to valid knight moves and non-repetition. Analyze how these constraints reduce possible moves at each step. Develop a backtracking-based solution to explore paths. Apply pruning to eliminate dead-end paths. Justify how your solution ensures completeness and discuss complexity.
Your Code
def is_safe(x, y, board, n):
return 0 <= x < n and 0 <= y < n and board[x][y] == -1
moves = [
(2, 1), (1, 2), (-1, 2), (-2, 1),
(-2, -1), (-1, -2), (1, -2), (2, -1)
]
def count_moves(x, y, board, n):
count = 0
for dx, dy in moves:
nx = x + dx
ny = y + dy
if is_safe(nx, ny, board, n):
count += 1
return count
def solve(x, y, move, board, n):
if move == n * n:
return True
next_moves = []
for dx, dy in moves:
nx = x + dx
ny = y + dy
if is_safe(nx, ny, board, n):
degree = count_moves(nx, ny, board, n)
next_moves.append((degree, nx, ny))
next_moves.sort()
for degree, nx, ny in next_moves:
board[nx][ny] = move
if solve(nx, ny, move + 1, board, n):
return True
board[nx][ny] = -1
return False
n = int(input())
board = [[-1] * n for _ in range(n)]
board[0][0] = 0
if solve(0, 0, 1, board, n):
print("Knight's tour:")
for row in board:
print(*row)
else:
print("No solution exists")
Input
8
Output
Knight's tour:
0 15 30 39 2 17 32 55
29 38 1 16 41 54 3 18
14 31 40 45 52 19 56 21
37 28 47 42 57 44 53 4
Submitted